

How This Project Works

The Complete Tour of the Pokémon-Playing AI Agent

June 2026

Abstract

This document explains everything: what the program does, how the AI “sees” and “plays” the game, what each piece of code is responsible for, and what actually happens step by step when you run it. No prior knowledge of the codebase is assumed.

Contents

1	The Big Picture	2
2	The Cast of Characters (the Modules)	2
3	How the AI “Sees” the Game	3
3.1	The Screenshots (Per-Press Keyframes)	3
3.2	The Memory Cheat-Sheet	3
3.3	The Explored World Map (Overworld Only)	4
4	How the AI “Acts”: Tools	5
4.1	Tool 1: <code>press_buttons</code> (the Main One)	5
4.2	Tool 2: <code>navigate_to</code> (On by Default)	5
5	The Brain Wiring: the LangGraph Flow	6
5.1	The Shared Clipboard: <code>GameAgentState</code>	6
5.2	Node by Node	7
5.3	The Routing Decisions (the Arrows)	7
6	The Conversation Itself (What the AI Literally Receives)	7
6.1	The <code>VisionToolNode</code> Trick	8
7	Summarization: the Memory Diet	8
8	Two Full Scenarios, Frame by Frame	9
8.1	Scenario A: a Normal Step (Talking to Professor Oak)	9
8.2	Scenario B: Hitting the History Limit Mid-Game	9
9	The Connection to the AI: <code>llm.py</code>	9
10	Configuration: <code>settings.py</code> and <code>.env</code>	10
11	Money: What a Run Costs	10
12	Honest Limitations (So You Are Not Surprised)	10

1 The Big Picture

This project makes an AI language model (LLM) play **Pokémon Red** by itself.

Think of it like this: imagine you are playing Pokémon over a video call with a friend. Your friend cannot touch the Game Boy — they can only:

1. **Look** at a photo of the screen you send them,
2. **Read** a little cheat-sheet you write for them (“you’re in Pallet Town, you have \$3000, no Pokémon yet, there’s a dialog box open…”),
3. **Tell you** which buttons to press (“press A twice, then walk up”).

You press the buttons for them, take a new photo, write a new cheat-sheet, and send it back. Repeat forever.

In this project:

- The “friend” is **Gemini Flash Lite** (an LLM, called through OpenRouter).
- “You” (the one holding the Game Boy) is the **Python program**.
- The Game Boy itself is **PyBoy**, a Game Boy emulator that runs the actual Pokémon Red ROM file on your computer.

One full exchange — *model looks* → *model decides* → *buttons get pressed* → *new screenshot taken* — is called a **step**. You choose how many steps to run:

```
uv run main.py --rom "Pokemon Red.gb" --steps 20
```

2 The Cast of Characters (the Modules)

```
pokemon/
|- main.py                <- entry point: wires everything, starts the loop
|- prompts/              <- the "instructions" given to the AI (Jinja)
|  |- system/game_player.j2 <- "You are playing Pokemon Red..."
|  |- human/summarize_request.j2 <- "Please summarize your progress..."
|  `-- human/summary_handoff.j2 <- "Here is your summary, continue playing..."
`- src/pokemon_agent/
   |- core/
   |  |- settings.py      <- all configuration (model name, temperature...)
   |  `-- llm.py          <- builds the connection to the AI model
   |- agent/
   |  |- graph.py         <- THE BRAIN WIRING: the loop logic (LangGraph)
   |  |- state.py         <- the shared "memory" of the loop
   |  |- tools/press_buttons.py <- the AI's "hands" (button presses)
   |  |- tools/navigate_to.py <- auto-walking tool (A* over the explored map)
   |  |- vision_tool_node.py <- makes screenshots visible to the AI
   |  `-- prompt_manager.py <- loads & renders the prompt templates
   `- emulator/
      |- emulator.py      <- wraps PyBoy: buttons, screenshots, maps
      |- memory_reader.py <- reads the game's RAM and decodes it
      `-- screenshot.py   <- converts screen images to base64 text
```

A quick one-liner for each:

Piece	In plain words
main.py	“Start the Game Boy, build the brain, run N steps.”
settings.py	A single place for every knob (which model, how hot, how many steps...). Reads your <code>.env</code> file.
llm.py	Creates the phone line to Gemini via OpenRouter, with retry logic for when the line is bad.
graph.py	The flowchart the program follows: think $\rightarrow$ act $\rightarrow$ observe $\rightarrow$ repeat $\rightarrow$ summarize when memory is full.
state.py	The clipboard everyone shares: message history, step counter, the emulator itself.
tools/	The two actions the AI is allowed to take.
vision_tool_node.py	The trick that lets the AI literally <i>see</i> screenshots inside tool results.
emulator.py	Knows how to press buttons, take screenshots, and maintain the explored world map.
memory_reader.py	Knows <i>where in the Game Boy’s RAM</i> the game stores money, party, dialog... and decodes it.

3 How the AI “Sees” the Game

The AI never touches pixels or RAM directly. After every action, the program builds an **observation** with three parts (see `agent/tools/_observation.py`):

3.1 The Screenshots (Per-Press Keyframes)

PyBoy exposes the current screen as a 160×144 pixel image. The emulator captures one **keyframe after each button press settles**, so a batched action (say, five A-presses through a dialog) yields five screenshots — the model sees everything that happened during its action, not just the end state (the last `MAX_KEYFRAMES` are kept, the final one always being the current state). Each frame is encoded as a **base64 data URL** — the image converted into a long text string like `data:image/png;base64,iVBORw0KGg...` — because LLM APIs only accept text/JSON. Token cost per frame is controlled by the `MEDIA_RESOLUTION` setting (Gemini-only; `low` $\approx$ 255 tokens/frame reads Game Boy text perfectly, verified empirically).

3.2 The Memory Cheat-Sheet

The Game Boy stores everything about the game in its RAM, and for a famous game like Pokémon Red, fans have documented **exactly which RAM address holds what**. `memory_reader.py` is a big collection of those addresses plus decoders. Examples:

RAM address	Contains	Quirk
0xD349	Your money	Stored in “Binary Coded Decimal” — each half-byte is one decimal digit
0xD35E	Current location ID	A number 0–255 mapped to names (“PALLET TOWN”, “VIRIDIAN CITY”...)
0xD163	Party size	How many Pokémon you carry
0xD16B+offsets	Pokémon #1 data	Species, HP, level, moves, status...
0xD356	Badges	One bit per badge, 8 badges in one byte
0xC3A0–0xC507	On-screen text	Pokémon uses its own character encoding (<code>0x80</code> = ‘A’, <code>0xA0</code> = ‘a’, ...) which we translate to normal text

The result is a human-readable block the AI receives as text:

Player: RED
Rival: BLUE
Money: \$3000
Location: PALLET TOWN
Coordinates: (5, 6)
Valid Moves: up, down, left
Badges: BOULDER
Inventory:
 POTION x2
Dialog: None

Pokemon Party:
CHARMANDER (CHARMANDER):
Level 8 - HP: 21/25
Types: FIRE
- SCRATCH (PP: 35)
- GROWL (PP: 40)

This matters because vision models are decent but not perfect at reading tiny Game Boy text — memory reading gives the AI *ground truth*.

3.3 The Explored World Map (Overworld Only)

The model has no spatial memory of its own: each turn it only sees the current screen, so left alone it re-explores the same rooms and circles mazes for hours (every public “LLM plays Pokémon” project hit this). So the program keeps a persistent **fog-of-war map**: after every settled button press, the walkability of every tile currently on screen is recorded into a per-map dictionary, in the game’s own coordinate system. The observation then includes a rendering of everything explored so far:

VIRIDIAN POKECENTER - explored map. You are at (x=3, y=7), facing up.

```
      0      5
y=3 #...S...
y=4 ##.....
y=5 ##.....
y=6 ###...##.
y=7 ###.^D.##.
y=8 #####
```

Legend: . explored ground, # wall/obstacle, ? unexplored,
D door/warp, S person/object standing there right now, arrow = you.
Doors/warps on this map: (3, 7), (4, 7)
People/objects on screen: (4, 3)
Reachable UNEXPLORED spots: (3, 3) 4 steps, (2, 3) 5 steps, ...

Three parts matter:

- **The map itself** remembers terrain far off screen, with coordinate rulers so the model can name any tile.
- **Doors/warps** come straight from the map header in RAM (wNumberOfWarps at 0xD3AE) — ground truth for where the exits are.
- **The frontier list** (“reachable UNEXPLORED spots”) is computed by flood-filling the explored map: walkable tiles that border unknown space. This is the single most effective anti-stuck signal reported by the Gemini Plays Pokémon project — when the model feels lost, the prompt tells it to go to one of these instead of claiming it is stuck.

The screen is read through PyBoy’s “game wrapper” (18×20 tiles, shrunk to 9×10 walkable squares since the player occupies 2×2 tiles) and anchored to the player’s RAM coordinates. Recording is skipped whenever the screen is not a clean overworld view (dialogs and menus overwrite the tiles they cover; mid-warp the RAM is half-updated). Tiles under NPCs stay unknown — NPCs move. The map is saved as `states/game-NNN.map.json` next to each save state, so resuming a game resumes its exploration memory too.

4 How the AI “Acts”: Tools

LLMs can do **tool calling** (a.k.a. function calling): you describe functions to the model in JSON, and instead of just answering with text, the model can reply “call `press_buttons` with `{"buttons": ["a"]}`”. The program then actually runs that function and sends the result back.

4.1 Tool 1: `press_buttons` (the Main One)

What the model sends:

```
{
  "say": "Time to pick our starter!",
  "buttons": ["up", "up", "a"]
}
```

- **say:** 1–2 sentences spoken aloud to the audience via TTS
- **buttons:** a sequence from the 8 Game Boy buttons (a, b, start, select, up, down, left, right)

What actually happens (in `emulator.py`): for each button, PyBoy holds the button for 10 frames, releases it, then runs the game until it *settles*. Settle detection polls every 8 frames and declares the action finished when (a) the game engine reports idle (walk counter zero, no scripted inputs — documented RAM flags from the poked disassembly) AND (b) the game’s own screen-composition buffer (`wTileMap` at `0xC3A0`, with the blinking masked) *plus the palette registers* (screen fades are palette-only — without them a mid-warp fade looks settled while the new map is half-loaded) stays byte-identical for 4 consecutive samples (10 samples when the textbox area is blank, the signature of a scene-transition pause). A 400-frame ceiling covers states that never go idle (cutscene walks, battle intros). Measured against the old fixed 120-frame wait: walking settles in ~ 40 frames ($3\times$ faster), menus faster still, and dialogs wait exactly as long as the text takes to print — the fixed wait used to capture half-printed dialog $\sim 16\%$ of the time; settle detection captures complete text.

What the model gets back:

```
{
  "result": "Pressed buttons: up, up, a",
  "memory_info": "Player: RED\nMoney: $3000\nLocation: ...",
  "map": "VIRIDIAN CITY - explored map. You are at (x=23, y=28)..."
}
```

plus one screenshot per button press, as actual images in press order (see Section 6 — this is the VisionToolNode trick).

4.2 Tool 2: `navigate_to` (On by Default)

Instead of pressing one arrow at a time, the model can say “walk me to (23, 25)” — using the same coordinates the explored map, the doors list, and the frontier list are labeled with. The

program runs **A*** over the fog-of-war map (not just the visible screen), so it can walk to places far off screen, then executes the path one verified step at a time:

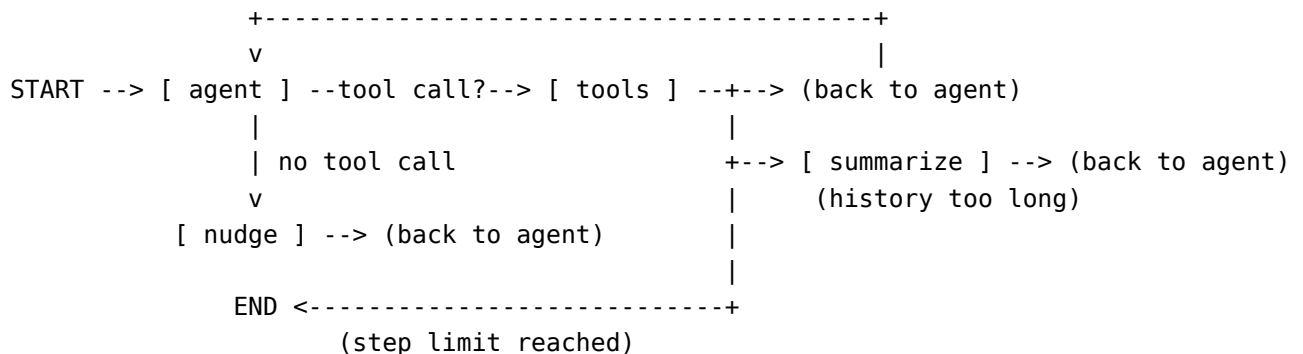
1. Plan a path through explored, walkable tiles to the target.
2. Press one step; check the player's RAM position actually changed.
3. If it didn't (a wandering NPC, a ledge, a spin tile — things collision data can't see), mark that step as blocked and re-plan around it.
4. Stop early — with an explanation to the model — when a warp triggers ("walked through a door into a new area"), when a dialog or battle interrupts, or when no route through explored terrain exists ("explore toward it first").

Targeting a door walks through it; targeting a person walks up and faces it (the final "bump" is how you talk to NPCs). Disable with `USE_NAVIGATOR=false` in your `.env`.

Why is this safe to leave on now? The old version could only path within the visible 9×10 window, which made it nearly useless. Long-range navigation over remembered terrain is the feature that, per the public Gemini-vs-Claude comparison, let Gemini get twice as far on a third of the actions.

5 The Brain Wiring: the LangGraph Flow

The loop is built with **LangGraph**, a library where you define your agent as a flowchart ("graph") of **nodes** (boxes that do work) connected by **edges** (arrows, possibly conditional). The flowchart here:



5.1 The Shared Clipboard: **GameAgentState**

Every node reads and writes one shared state object:

```

{
  "messages": [...],          # the full conversation with the AI so far
  "emulator": <Emulator>,     # the live Game Boy (tools grab it from here)
  "step_count": 3,            # how many actions taken so far
  "max_steps": 10,            # stop after this many
  "max_history_tokens": 30000, # summarize when a prompt reaches this size
}

```

`messages` is special: it uses LangGraph's `add_messages` rule, meaning nodes do not overwrite the list — they *append* to it (or delete from it with special "remove" markers, used by summarization).

5.2 Node by Node

agent — sends the whole conversation to the model and gets back its decision. By default (`FORCE_TOOL_USE=false`) the tools are bound with `tool_choice="auto"`, so the model may narrate its reasoning before acting; a reply without any tool call is handled by the **nudge** node. Setting `FORCE_TOOL_USE=true` switches to `tool_choice="any"`, which **forces** a tool call every turn — never stalls, but (by API design) suppresses all visible reasoning text. If the reply contains a tool call, the step counter goes up by one.

tools — actually executes the tool call (presses the buttons), captures the observation (screenshot + memory + explored map), and appends it to the conversation as a “tool result” message.

summarize — see Section 7.

nudge — handles replies with no tool call (a tool-less reply means nothing happened in the game). This node appends a human message — *“You did not call a tool in your last reply. Nothing happened in the game...”* (from `prompts/human/nudge_no_tool.j2`) — and sends the conversation back to the agent for another try. The message matters for two reasons: it keeps the conversation alternating user/assistant (providers dislike two assistant messages in a row), and it gives the model an explicit signal to break out of a “talking instead of acting” pattern. There is no retry cap — the run only ends at the step limit; LangGraph’s `recursion_limit` (set in `main.py`) is the final cost backstop if a model never acts.

With the default `FORCE_TOOL_USE=false` (`tool_choice="auto"`), the nudge is the active stall-prevention. With `FORCE_TOOL_USE=true`, tool calls are forced at the API level and this node is nearly dead code (it would only fire on provider quirks or truncated replies).

5.3 The Routing Decisions (the Arrows)

After **agent**: did the reply contain a tool call? Yes → go to **tools**. No → **nudge**, then back to the agent. The agent side never ends the run.

After **tools**, three checks in order:

1. `step_count >= max_steps?` → **END** (we are done; you asked for N steps).
2. Did the last turn’s prompt reach `max_history_tokens`? → **summarize** (conversation getting too fat). The size comes from the provider’s reported token usage — the exact measured size of the conversation, images included — not from counting messages or running a local tokenizer. (If a provider returns no usage data, a 30-message count is the fallback trigger.)
3. Otherwise → back to **agent** for the next step.

6 The Conversation Itself (What the AI Literally Receives)

LLMs are stateless — they remember *nothing* between calls. Every single step, we send the **entire conversation from the beginning**. The conversation looks like:

```
[SYSTEM] You are playing Pokemon Red... (from prompts/system/game_player.j2)
[USER]    You may now begin playing.
[AI]      I'll start by pressing A to begin. -> press_buttons(["a"])
[TOOL]    {"result": "Pressed buttons: a", "memory_info": ...} + <screenshot>
[AI]      A dialog is open, I'll continue. -> press_buttons(["a"])
[TOOL]    {"result": ...} + <screenshot>
[AI]      ...                               -> press_buttons(["a", "a"])
...
```

Reality check: how much useful text the [AI] line contains depends on the model and on `FORCE_TOOL_USE`. Observed in real runs:

- `FORCE_TOOL_USE=true (tool_choice="any")`: *no* text at all, from any model — forced tool use suppresses text by API design. The conversation is [AI] "" -> `press_buttons(...)`.
- `auto` + Claude Haiku 4.5: proper narration every turn (“I see the main menu with NEW GAME and OPTION. I’ll select NEW GAME...”).
- `auto` + Gemini Flash Lite: acts reliably but emits only stray token fragments (“elf”, “thought”) instead of real explanations — a quirk of small Gemini models when mixing text with function calls. Harmless, just not informative.

A mandatory “reasoning” parameter in the tool schema would force explanations out of any model, at the cost of a few tokens per turn.

6.1 The VisionToolNode Trick

There is a subtlety here. In the OpenAI-style API format, **tool result messages normally carry only text** — no images. The naive workaround (which this project used before the refactor) was to send the screenshot as a separate follow-up user message, which clutters the conversation.

`VisionToolNode` does it properly. When a tool returns a dictionary, it checks for image-ish keys (like “screenshot”) containing a `data:image/...` string, and splits the tool message into **content blocks**:

```
[
  {"type": "text",      "text": "{\\"result\\": \\"Pressed buttons: a\\", ...}"},
  {"type": "image_url", "image_url": {"url": "data:image/png;base64,iVBOR..."}}
]
```

LangChain supports image blocks inside tool messages, and OpenRouter translates this correctly for Gemini. So the screenshot rides *inside* the tool result, exactly where it belongs.

7 Summarization: the Memory Diet

The problem: every step adds ~2 messages, and each tool message contains a full screenshot (~50–100 KB of base64 text). After 15 steps you are sending megabytes of history on *every* call. Cost grows, speed drops, and eventually you would blow past the model’s context limit.

The fix: when a turn’s prompt reaches `max_history_tokens` (default 30,000, measured from the provider’s reported usage), the `summarize` node:

1. Sends the whole conversation to the model with one extra request: *“Summarize your progress: key events, decisions, current objectives, location, team status, plans.”* (from `prompts/human/summarize_request.j2`)
2. Wipes the **entire** message history (a special `REMOVE_ALL_MESSAGES` marker).
3. Rebuilds a tiny fresh conversation:
 - the same system prompt,
 - one user message: *“CONVERSATION HISTORY SUMMARY (representing 30 previous messages): ...”* + a **fresh screenshot** so the model is not blind + *“You may now continue playing.”*

The model loses the play-by-play details but keeps the plot. It is like a “previously on...” TV recap. Then the loop continues normally.

8 Two Full Scenarios, Frame by Frame

8.1 Scenario A: a Normal Step (Talking to Professor Oak)

1. **agent node** → the entire conversation (system prompt + history) is POSTed to OpenRouter. Gemini sees the last screenshot showing a dialog box: *“OAK: Your very own POKÉMON legend is about to unfold!”*
2. Gemini replies: text reasoning *“There’s an active dialog, I should advance it”* + tool call `press_buttons(buttons=["a"], wait=true)`.
3. **Router**: tool call exists → go to **tools**.
4. **tools node** → grabs the emulator from state; PyBoy holds A for 10 frames, releases, then runs until the game settles (the dialog finishes printing). Then it captures: new screenshot (base64), memory dump (**Dialog**: *“So! You want the fire POKÉMON, CHARMANDER?”*), no explored map (we are in a dialog, not the overworld).
5. The observation becomes a multimodal tool message appended to the conversation. Step counter: 7 → 8.
6. **Router**: $8 < \text{max_steps}$, 18 messages < 30 → back to **agent**. Repeat.

8.2 Scenario B: Hitting the History Limit Mid-Game

1. Step 14 finishes; the last prompt weighed in at 31,000 tokens. Router after **tools**: step limit not reached, but $31,000 \geq \text{max_history_tokens}$ → go to **summarize**.
2. The summarize node asks the model for a recap. Gemini returns: *“1. Key events: started the game, chose Charmander... 4. Currently on Route 1 heading to Viridian City...”*
3. History is wiped. New history = system prompt + recap + fresh screenshot + “continue playing”. That is 2 messages instead of 30.
4. Back to **agent**: Gemini reads the recap, looks at the screenshot, and carries on walking north — without remembering each individual button press, but knowing exactly where it is and why.

9 The Connection to the AI: `llm.py`

We talk to **OpenRouter** — a broker that exposes hundreds of models behind one OpenAI-compatible API. Which model is just a setting (`GAME_MODEL=google/gemini-3.1-flash-lite`), so you can swap to Claude or GPT by editing `.env`, no code changes.

`get_llm()` builds a **RobustChatOpenAI** — a hardened version of LangChain’s standard OpenAI client (ported from the `tolatex` backend) that handles real-world OpenRouter problems:

- **Garbage in a 200 response**: sometimes the HTTP call “succeeds” but the body is truncated JSON, or contains an embedded error (`{"message": "Provider returned error", "code": 500}`). It detects these and retries with growing pauses (3s, 9s, 27s).
- **Infinite generation loops**: if the model gets stuck repeating itself until it hits the token ceiling (`finish_reason: "length"`), it can detect this and retry (off by default).
- **Reasoning preservation**: “thinking” models return their reasoning in nonstandard fields that normally get dropped; this client saves them and re-sends them on later turns so multi-turn thinking stays coherent. With `THINKING=none` (the default here) this is dormant, but flip `THINKING=low/medium/high` in `.env` and it activates.
- **Provider sorting**: requests OpenRouter’s lowest-latency provider for the model.

10 Configuration: `settings.py` and `.env`

All knobs live in one `pydantic-settings` class. Anything can be set in `.env` (or as a real environment variable, case-insensitive):

Setting	Default	Meaning
<code>OPENROUTER_API_KEY</code>	—	your key (required)
<code>GAME_MODEL</code>	<code>google/gemini-3.1-flash-lite</code>	the model that plays
<code>SUMMARIZER_MODEL</code>	<code>""</code> (= game model)	the model that writes recaps
<code>TEMPERATURE</code>	1.0	randomness of decisions
<code>MAX_TOKENS</code>	4000	max length of each model reply
<code>THINKING</code>	<code>none</code>	reasoning effort: none/minimal/low/medium/high
<code>MAX_STEPS</code>	10	default number of steps (CLI <code>--steps</code> overrides)
<code>MAX_HISTORY_TOKENS</code>	30000	prompt size that triggers a summary (CLI <code>--max-history-tokens</code> overrides)
<code>MAX_KEYFRAMES</code>	8	per-press screenshots attached to an observation (most recent kept)
<code>MEDIA_RESOLUTION</code>	low	Gemini image token spend per frame (low $\approx$ 255, high $\approx$ 1085)
<code>USE_NAVIGATOR</code>	true	expose the A* <code>navigate_to</code> tool
<code>FORCE_TOOL_USE</code>	false	<code>true</code> forces a tool call every turn but suppresses reasoning text
<code>ROM_PATH</code>	<code>Pokemon Red.gb</code>	default ROM file (CLI <code>--rom</code> overrides)
<code>MAX_REQUESTS_PER_SECOND</code>	0 (off)	global API rate limit

`get_settings()` is cached — the `.env` is read once per run. The prompts are equally swappable: they are Jinja2 template files in `prompts/`, so you can tweak the AI’s personality or goals without touching Python.

11 Money: What a Run Costs

Each step sends the full history, so cost grows until a summary resets it. With Gemini Flash Lite, observed costs were roughly **\$0.0001–0.0025 per step** (rising as history grows), so a 10-step run is about a penny. The two real cost levers:

- `MAX_HISTORY_TOKENS` — lower = more frequent summaries = cheaper but more forgetful.
- The model — flash-lite is among the cheapest vision models; switching to a frontier model multiplies cost by $\sim 100\times$.

12 Honest Limitations (So You Are Not Surprised)

- **One action per turn, no multi-reply deliberation** — the loop expects each reply to act (tool-less replies just get nudged); the model cannot “stop and think” across several

replies before committing.

- **Its spatial memory is only as good as its exploration:** the fog-of-war map and `navigate_to` cover everything it has *seen*, but cross-town trips through never-visited territory still mean exploring tile by tile, guided by the frontier list and its imperfect world knowledge of Pokémon Red.
- **Summaries lose detail:** if something important was not mentioned in the recap, it is gone (e.g., “I already searched that house”).
- **Flash-lite is cheap, not brilliant:** it tends to skip the “explain your reasoning” part and just mash A through dialogs. It works, but do not expect speedrun strategy. Swap in a stronger model for smarter play.
- **No save/checkpoint logic in the agent** — the emulator supports loading a saved state (`--load-state file`), but the agent does not auto-save progress.